KLAi

Information theory: entropy, cross-entropy, KL divergence, softmax and perplexity

How does a language model turn raw scores into a probability for every next word, and how do we score it?

Mathematics for AI and Machine Learning | Session 17 of 18 | Probability and statistics

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Compute surprise, entropy, cross-entropy and KL divergence by hand, and show that cross-entropy = entropy + KL



Compute the softmax of three scores and derive that the gradient of softmax with cross-entropy is prediction minus target



Compute the cross-entropy and perplexity of a toy language model on a six word sentence

Your two hours: information, softmax, then perplexity

0:00	10	Recall, then a real question from AI
0:10	35	Concept: a picture, then one example by hand
0:45	10	Break
0:55	45	Lab in Python or a spreadsheet: predict, run, change one thing
1:40	15	Where this appears in AI, and an exit check
1:55	5	Save the work

Ideas first. Then the lab, in Python or a spreadsheet. Then AI.

Three questions from last time

1. What does maximum likelihood estimation choose?

2. Why do we work with the log-likelihood?

3. Which assumption about the target gives cross-entropy as the loss?



Answer aloud first. Then we check together.

Answers to the three questions from Session 16

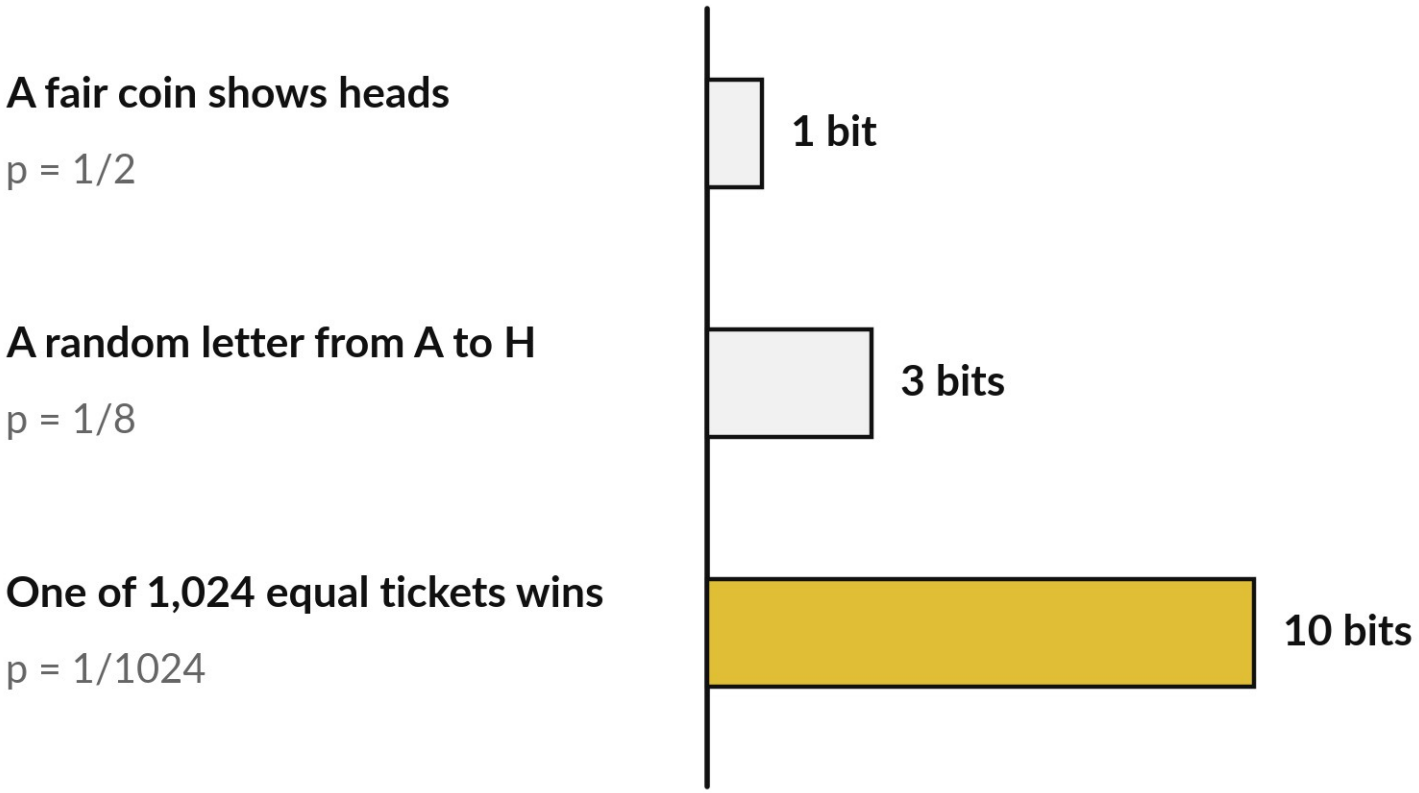
Question	Short answer
What does maximum likelihood estimation choose?	The parameter value that makes the observed data most probable, which means the largest likelihood.
Why do we work with the log-likelihood?	A product of many small probabilities underflows to zero. A sum of logarithms is stable, and the maximum stays at the same parameter.
Which assumption about the target gives cross-entropy as the loss?	A Bernoulli assumption for yes or no, or a categorical assumption for one of many classes. Gaussian noise gives squared error.

Today the Bernoulli and categorical losses get a new name and a new meaning.

Words we use today

	Word	What it means
	**Entropy**	The average surprise of a distribution, in bits. How uncertain the outcome is.
	**Cross-entropy**	The average surprise when reality follows P but the model believes Q. The training loss for classification and language models.
	**KL divergence**	Kullback-Leibler divergence: cross-entropy minus entropy. The extra cost of believing Q instead of P.
	**Softmax**	A function that turns scores (logits) into probabilities that are positive and add to 1.
	**Perplexity**	e to the power of the mean negative log-probability. The effective number of equally likely choices the model has at each word.

Surprise: a rare event carries more information than a common one



The surprise (self-information) of an event with probability p is -log2(p) bits.

A bit is the answer to one yes or no question.

Every time the probability is cut in half, the surprise grows by one bit.

Surprise is the negative logarithm of the probability

Surprise in bits

$$I(x) = -\log_2 p(x)$$

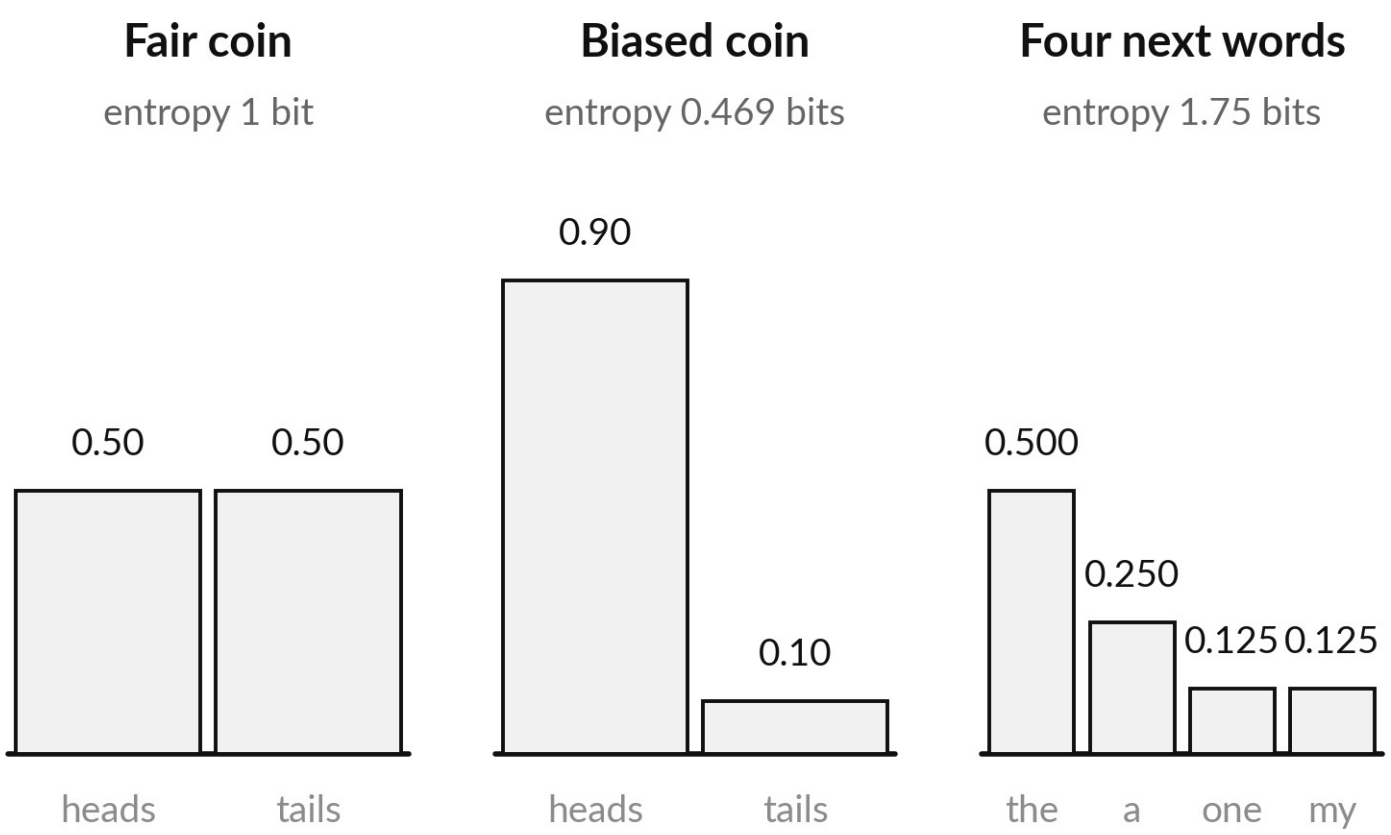
Surprise in nats

$$I(x) = -\ln p(x)$$

Symbol	What it means	In our data
x	One outcome, such as one word.	a random letter from A to H
p(x)	The probability of that outcome.	1/8
I(x)	The surprise (self-information) of that outcome.	3 bits = 2.079 nats
nat	The unit when we use ln. One bit is 0.693 nats.	libraries use ln

Libraries usually use ln, so their losses are in nats. Bits and nats differ by a constant factor.

Entropy is the average surprise, and it is largest when the outcome is hardest to guess



Entropy H(P) is the average surprise of outcomes drawn from the distribution P.

A fair coin is the hardest to predict: 1 bit.

A coin that gives heads 90 percent of the time has only 0.469 bits.

By hand: the entropy of four next words is the sum of p times surprise, 1.75 bits

Next word	p	Surprise (bits)	p × surprise
the	0.5	1	0.5
a	0.25	2	0.5
one	0.125	3	0.375
my	0.125	3	0.375
Total	1		1.75

Illustrative (made-up) probabilities. The total of the last column is the entropy.

Entropy is the probability-weighted average of the surprise

Entropy

$$H(P) = -\sum_i p_i \log_2 p_i$$

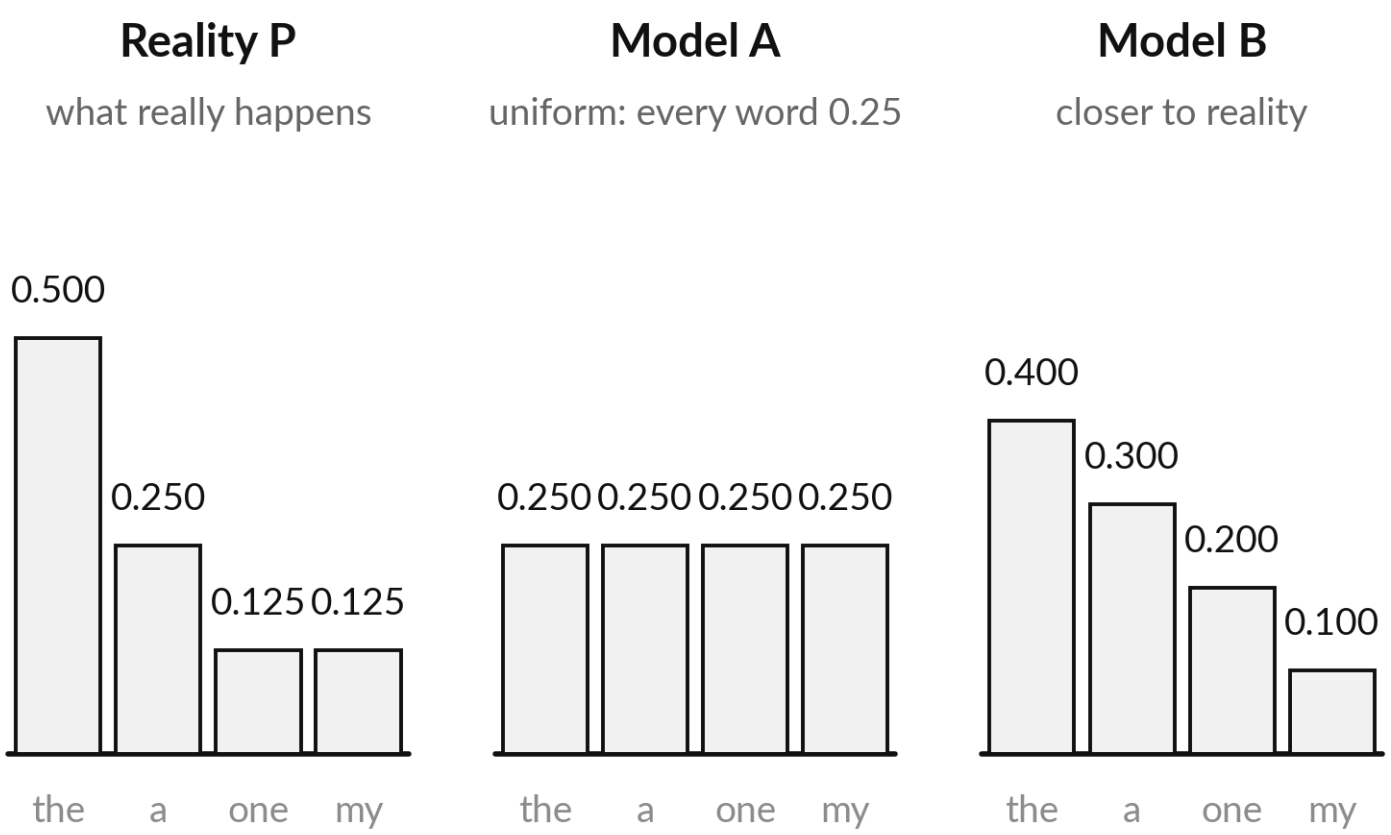
Our four words

$$H(P) = 0.5 + 0.5 + 0.375 + 0.375 = 1.75 \text{ bits}$$

Symbol	What it means	In our data
P	A distribution: probabilities that add to 1.	the four next words
p_i	The probability of outcome i.	0.5, 0.25, 0.125, 0.125
sum over i	Add the terms for every outcome.	four terms
H(P)	The entropy, in bits when the log is base 2.	1.75 bits

Four equally likely words would have the largest entropy: 2 bits.

Cross-entropy is the average surprise when reality follows P but the model believes Q



Cross-entropy H(P, Q) averages the model's surprise over what really happens.

Model A: H(P, Q) = 2.000 bits.

Model B: H(P, Q) = 1.801 bits. Smaller, because Q is closer to P.

Cross-entropy weights the model's surprise by the true probabilities

Cross-entropy

$$H(P, Q) = -\sum_i p_i \log_2 q_i$$

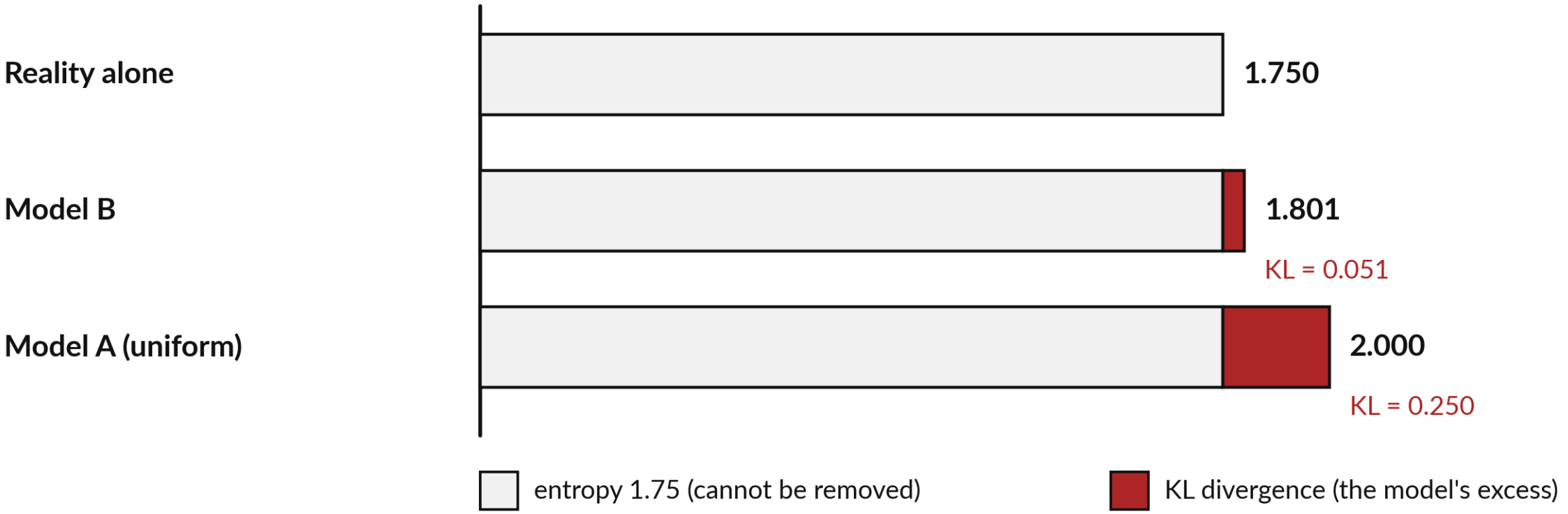
Model A

$$H(P, Q_A) = (0.5 + 0.25 + 0.125 + 0.125) \times 2 = 2.000$$

Symbol	What it means	In our data
p_i	The true probability of outcome i.	0.5, 0.25, 0.125, 0.125
q_i	The probability the model gives to outcome i.	A: 0.25 each
log2 q_i	The model's surprise at outcome i.	A: 2 bits for each
H(P, Q)	The cross-entropy.	A: 2.000, B: 1.801

With a real label, p is 1 on the true class and 0 elsewhere, so the loss is -log q of the true class.

Cross-entropy = entropy + KL divergence: the entropy is fixed, the KL is what the model can improve



Model A: 1.750 + 0.250 = 2.000. Model B: 1.750 + 0.051 = 1.801. All in bits.

KL divergence is never negative and is zero only when the model equals reality

KL divergence

$$D_{KL}(P \parallel Q) = \sum_i p_i \log_2 \frac{p_i}{q_i}$$

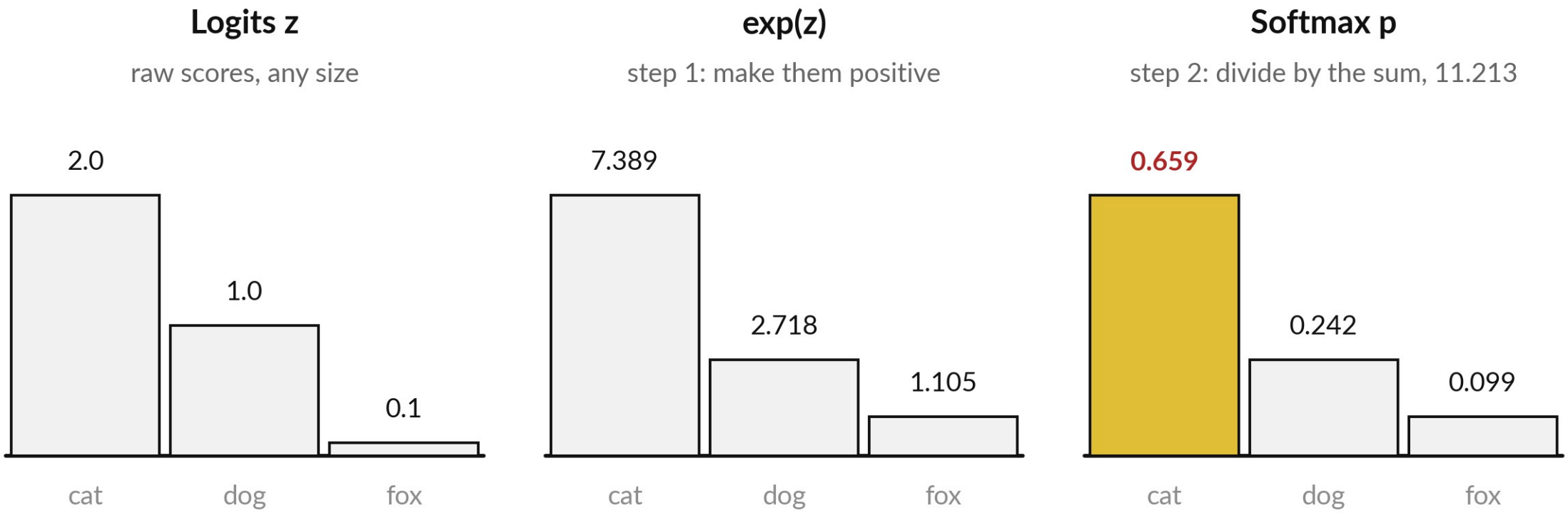
The link

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

Symbol	What it means	In our data
D_KL(P || Q)	The extra cost of believing Q when reality is P.	A: 0.250, B: 0.051 bits
Never negative	It is 0 only when Q = P everywhere.	a perfect model
Not symmetric	D(P || Q) is usually not D(Q || P).	so it is not a distance
H(P)	The entropy of reality.	1.75 bits

Minimising cross-entropy over the model is the same as minimising KL, because H(P) is fixed.

The softmax turns raw scores (logits) into probabilities that add to 1



Illustrative scores for three classes. The probabilities 0.659 + 0.242 + 0.099 add to 1.

The softmax formula, and how the temperature T reshapes it

Softmax

$$p_i = \exp(z_i) / (\exp(z_1) + \exp(z_2) + \cdots + \exp(z_K))$$

With temperature T

$$p_i = \exp(z_i/T) / (\exp(z_1/T) + \cdots + \exp(z_K/T))$$

Symbol	What it means	In our data
z_i	The logit (score) of class i.	2, 1, 0.1
p_i	The probability of class i.	0.659, 0.242, 0.099
K	The number of classes.	3
T	The temperature. Small T sharpens, large T flattens.	T = 0.5 gives 0.864 to the top class

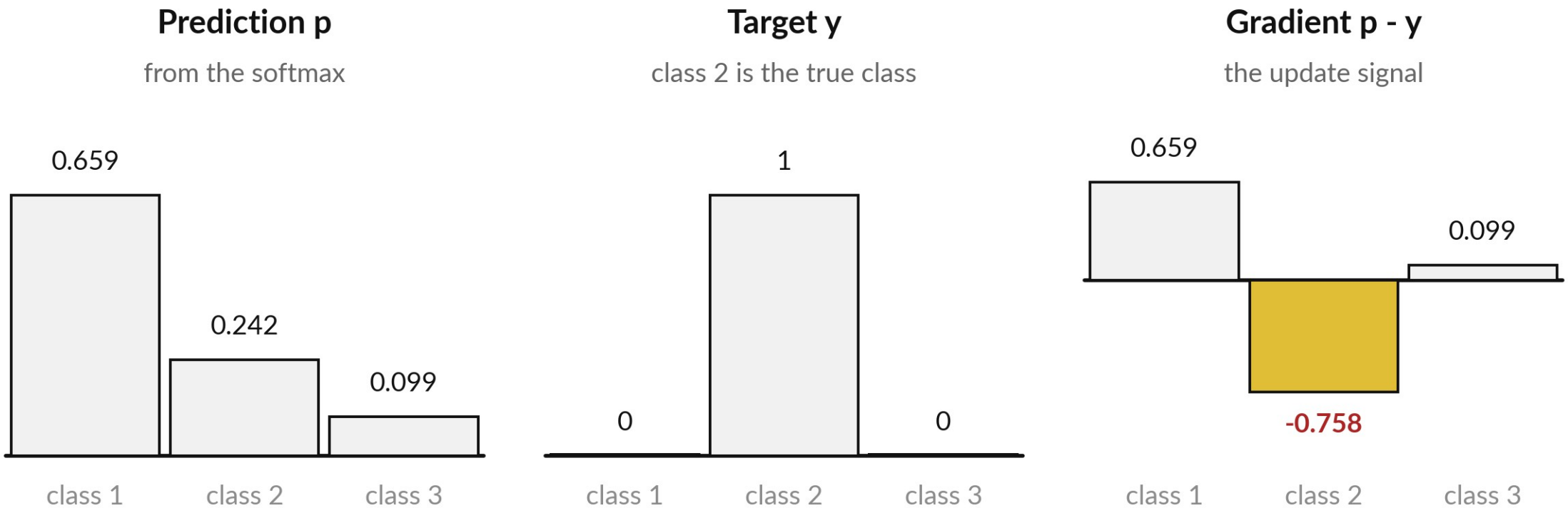
In the standard definition of temperature sampling, every logit is divided by T before the softmax.

Subtract the largest logit first, so large scores cannot overflow

Illustrative: three logits of 1000, 1001 and 1002. The softmax must give the same answer as for 0, 1 and 2.

Naive	e^1000 is too big for a computer, so it gives inf / inf = nan.
Shift	Subtract the largest logit, 1002: the scores become -2, -1, 0.
Exponent	e^-2 = 0.135, e^-1 = 0.368, e^0 = 1. Their sum is 1.503.
Result	0.135 / 1.503 = 0.090, then 0.245 and 0.665.

Softmax with cross-entropy has a simple gradient with respect to the logits: prediction minus target



Illustrative. The gradient for the true class is negative: its logit should go up.

Derivation: the gradient of softmax with cross-entropy is p minus y

Slopes

$$\frac{\partial L}{\partial p_k} = -\frac{y_k}{p_k}, \qquad \frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j)$$

Chain rule

$$\frac{\partial L}{\partial z_j} = -\sum_k \frac{y_k}{p_k} p_k(\delta_{kj} - p_j) = -\sum_k y_k(\delta_{kj} - p_j) = p_j - y_j$$

Symbol	What it means	In our data
L	The loss: L = -sum of y_k ln p_k.	1.417 in our example
delta_kj	1 when k = j, and 0 otherwise.	picks one term
y_k	The one-hot target: 1 for the true class.	y = (0, 1, 0)
p_j - y_j	The gradient for logit j.	(0.659, -0.758, 0.099)

The 1 / p_k cancels p_k. The delta part gives y_j. The other part gives p_j, because the y_k add to 1.

By hand: logits 2.0, 1.0, 0.1 with class 2 true give a loss of 1.417

Illustrative three-class example. The numbers match the lab.

Softmax, target	p = (0.659, 0.242, 0.099) and y = (0, 1, 0)
Loss	L = -ln 0.242 = 1.417 nats
Gradient	p - y = (0.659, -0.758, 0.099). The three entries add to 0.
Check	Add 0.001 to the first logit: the loss rises by about 0.00066.

Perplexity: e raised to the power of the mean negative log-probability of the real words

the next word	The	cat	sat	on	the	mat	total
probability the model gave it	0.2	0.1	0.25	0.5	0.4	0.05	
-ln(p)	1.61	2.30	1.39	0.69	0.92	3.00	9.90

Illustrative (made-up numbers): the probability a toy model gave to each real word of the sentence.

Mean -ln p = 9.90 / 6 = 1.651. Perplexity = e^1.651 = 5.21.

Perplexity is the exponential of the cross-entropy per token

Perplexity

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \ln p(w_i \mid w_1, \ldots, w_{i-1})\right)$$

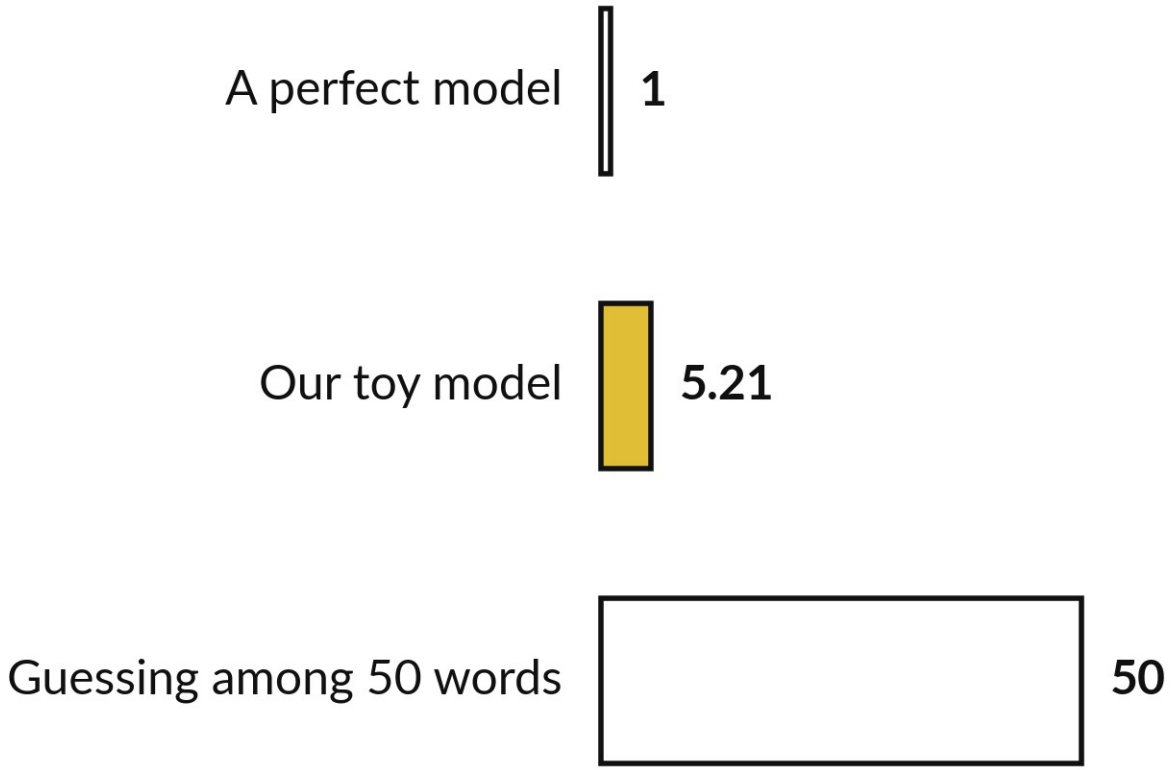
Our sentence

$$\mathrm{PPL} = e^{1.651} = 5.21$$

Symbol	What it means	In our data
N	The number of words (tokens) scored.	6
p(w_i | w_1 ... w_(i-1))	The probability the model gave to the real word i, given the words before it.	0.2, 0.1, 0.25, 0.5, 0.4, 0.05
Mean of -ln p	The cross-entropy per word, in nats.	1.651
PPL	Perplexity. In bits it is 2 to the power of the bits per word. Same value.	5.21

Lower is better. Perplexity 1 means the model gave probability 1 to every real word.

Perplexity reads as the effective number of equally likely words the model chooses between



A perplexity of 5.21 means the model is as unsure as if it picked among about five equally likely words.

Illustrative numbers.

Compare perplexities only for the same test text and the same tokens.

Three common mistakes about information measures

The mistake

Cross-entropy and KL divergence are the same thing.

Bits and nats give different perplexities.

Lower perplexity always means a better model.

The fix

They differ by the entropy of the data.

Same value if the exponent matches the log base.

Only on the same text and tokens.

Always say which log base, and which text.

Read the recipe before you write it: entropy, KL, softmax, gradient, perplexity

```
H = -np.sum(P * np.log2(P))
kl = np.sum(P * np.log2(P / Q))
p = np.exp(z - z.max()); p = p / p.sum()
grad = p - y
ppl = np.exp(np.mean(-np.log(token_probs)))
```

In plain words

Line 1: entropy in bits.

Line 2: KL divergence. Cross-entropy = H + kl.

Line 3: stable softmax: shift by the largest logit.

Line 4: the gradient with respect to the logits.

Line 5: perplexity from the word probabilities.

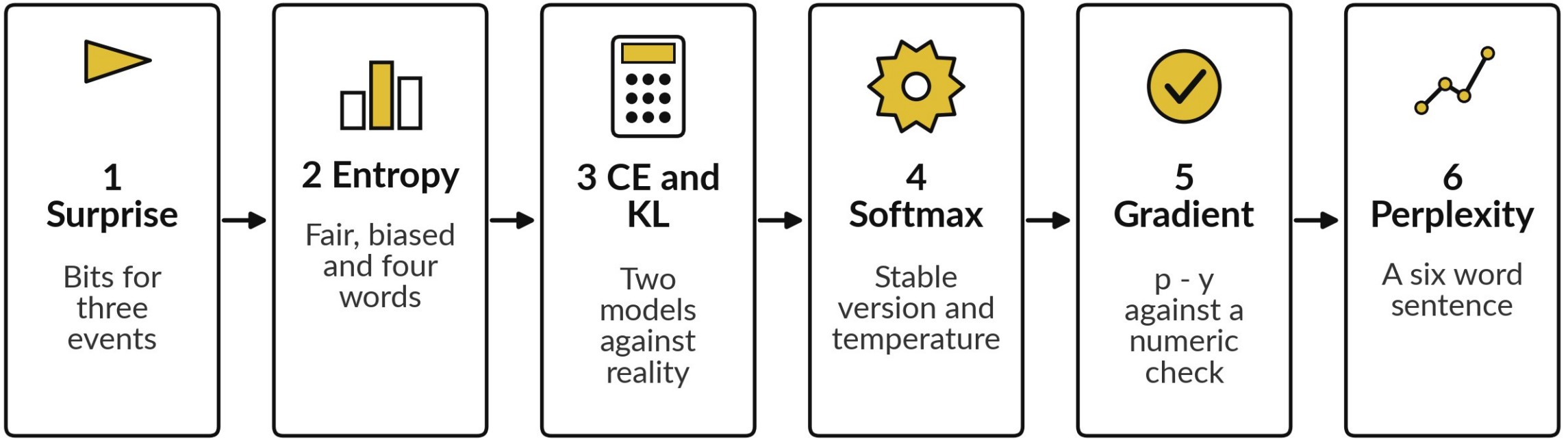
Spreadsheet: a column =LOG(q,2), SUMPRODUCT, and =EXP(AVERAGE(...)).



Part 2: the lab

Close the slides. Open the notebook or the workbook.

The lab has six steps: surprise, entropy, cross-entropy, softmax, gradient and perplexity



Python: Steps 1 to 6 in the notebook. Spreadsheet: sheets Entropy, Softmax and Perplexity with the same numbers. Predict, run, change one thing.

Check your numbers against these before you move on

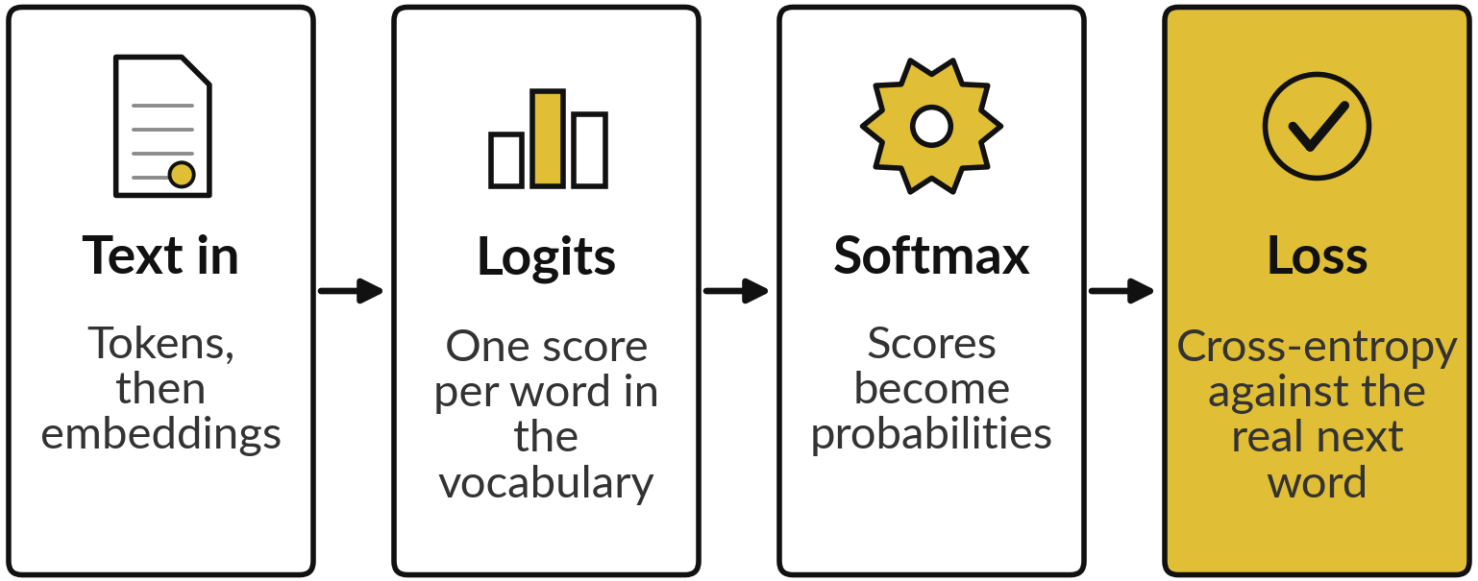
What you compute	You should see
Surprise of 1/2, 1/8 and 1/1024 in bits	1, 3, 10
Entropy of the four-word P, fair coin, 0.9 coin	1.75, 1, 0.469 bits
Cross-entropy of model A and model B	2.000 and 1.801 bits
KL divergence of model A and model B	0.250 and 0.051 bits
Softmax of 2.0, 1.0, 0.1	0.6590, 0.2424, 0.0986
Loss and gradient p - y (class 2 true)	1.4170; 0.6590, -0.7576, 0.0986
Sum of -ln p, mean, perplexity of the sentence	9.903, 1.6506, 5.210

If your number differs, stop and ask. Find the step that went wrong. Do not change a number to match.

Errors you will meet, what they mean and what to do

	What you see	What it means	What to do
	**nan from the softmax**	The exponential overflowed	Subtract the largest logit first
	**TypeError: NoneType**	An answer cell (marked gold) still says None	Replace None with your code and run again
	**A negative KL divergence**	P and Q are swapped, or they do not add to 1	Check both add to 1 and keep the order P then Q
	**Spreadsheet shows #DIV/0! or #NUM!**	A probability of 0 inside a logarithm	Use only probabilities above 0

Where this appears in AI: pre-training by minimising cross-entropy on the next token

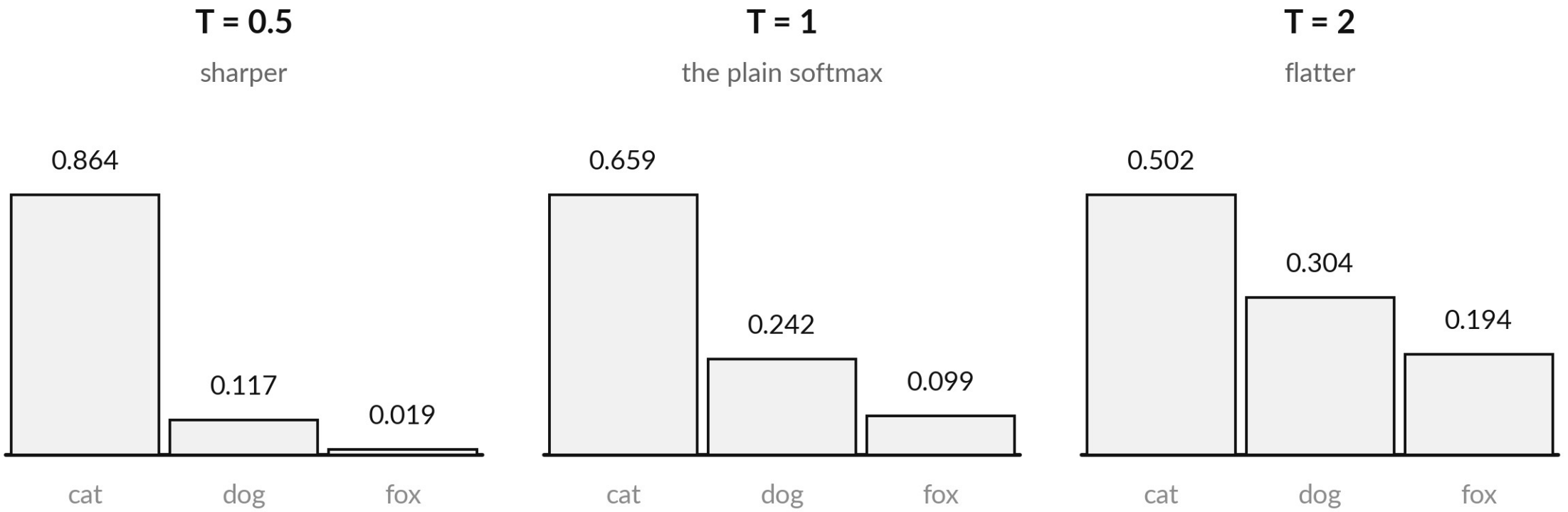


Published research describes transformer language models as pre-trained by minimising cross-entropy on the next token.

The p - y gradient from today is the signal that starts backpropagation.

Later stages, such as learning from human feedback, use other objectives. Closed products do not publish their internals.

Where this appears in AI: the temperature setting reshapes the softmax



Illustrative logits 2.0, 1.0, 0.1. Public APIs document temperature and top-p settings.

Check an AI assistant's answer against your own numbers

5 minutes Optional, with any assistant

Ask an assistant for the softmax of the logits 2.0, 1.0, 0.1 and the cross-entropy loss if class 2 is true.

Compare with 0.659, 0.242, 0.099 and a loss of 1.417 (natural log).

Ask which log base it used. Bits would give 2.04.

Never paste confidential data into an assistant.

You keep

A note: where the assistant agreed with your numbers, and where it did not.

Exit check: three questions and their short answers

Question	Short answer
What is the entropy of a fair coin, in bits?	1 bit.
What is the gradient of softmax with cross-entropy with respect to the logits?	The prediction minus the target: p - y.
A model has a mean negative log-probability of 2.0 nats per word. What is its perplexity?	e^2 = 7.39.

Answer each in one sentence before you read the right-hand column.

Save your lab and keep six numbers

What you keep from today

Keep your notebook or workbook with all checks passing. Keep six numbers: entropy 1.75 bits, cross-entropy 2.000, KL 0.250, softmax 0.659, gradient -0.758 and perplexity 5.21.

Our question today

How does a language model turn raw scores into a probability for every next word, and how do we score it?

Next session

Session 18: the capstone. Build linear regression, logistic regression, PCA, a small network and attention from first principles.

Take away

Cross-entropy = entropy + KL
2.000 = 1.750 + 0.250

Gradient
p - y = (0.659, -0.758, 0.099)

Toy perplexity
5.21

Next session
Capstone: five builds